

A Progressive Modeling Strategy for Bi-Objective BESS

Optimization:

A Phase II Design Rationale

Gen Li (Team SoloGen)

November 6, 2025

1 Executive Summary: The Strategic Framework

This report details the logical evolution of the Phase II Battery Energy Storage System (BESS) optimization model for the Huawei TechArena 2025. The primary challenge of Phase II is to transition from a single-objective (profit) model with rigid physical constraints to a sophisticated, bi-objective framework that maximizes the 10-Year Return on Investment (ROI) by dynamically trading market revenue against the economic cost of battery degradation.[1]

This design rationale is presented as a three-stage progressive narrative:

1. **Stage I (Baseline):** This model, based on Phase I, operates under a rigid and "economically blind" Daily Cycle Limit.[1] It is computationally trivial but economically sub-optimal, as it cannot differentiate between high- and low-value cycles.
2. **Stage II (Xu-Only):** This model introduces an economic degradation cost, replacing the rigid limit with a linearized *cyclic* aging cost function (C^{cyc}) based on the methodology of Xu et al. [1, 1]. This model is "economically smart" and computationally "light," as its non-linear cost approximation is inherently handled by linear programming logic.[1]
3. **Stage III (Xu + Collath):** This is the "academically complete" model, representing the full formulation in `p2_bi_model_ggdp.tex`. It adds a *calendar* aging cost (C^{cal}) based on Collath et al. [1, 1]. This addition, while physically accurate, is computationally "heavy," requiring the introduction of thousands of Special Ordered Sets of Type 2 (SOS2) variables, transforming the problem into a complex Mixed-Integer Linear Program (MILP) [1, 1].

The final model is executed via a **Two-Stage Meta-Optimization** strategy, as defined in our framework.[1] This strategy consists of:

- An **Outer Loop** (an α -sweep) to discover the optimal "degradation price" (α) that maximizes the 10-Year ROI.
- An **Inner Loop** (a 365-day Rolling Horizon/MPC simulation) that is called by the Outer Loop for *each* α value to determine the annual profit and degradation.

The central conclusion of this report is that the *compounding computational burden* of this meta-optimization strategy—which requires over 65,000 individual MILP solves—when combined with the competition's "open-source solver" constraint, renders the "complete" Stage III model computationally intractable.[1]

Therefore, this report provides the strategic justification for selecting **Stage II (Xu-Only)** as the final competition model. It delivers the optimal balance of physical realism and computational feasibility, capturing the dominant degradation driver (cyclic aging) while remaining solvable within the practical constraints of the competition.

Table 1: Model Stage Evolution and Trade-Off Analysis

Feature	Stage I (Phase I Baseline)	Stage II (Model ii: Xu-Only)	Stage III (Model iii: Xu + Collath)
Degradation Model	Rigid "Daily Cycle Limit" [1]	Economic: Cyclic-Only (C^{cyc}) [1]	Economic: Cyclic + Calendar ($C^{cyc} + C^{cal}$) [1]
Economic Logic	"Economically Blind" - hard stop	"Economically Smart" - trades profit vs. cyclic cost	"Physically Complete" - trades profit vs. cyclic & calendar cost
Key Methodology	N/A (Simple constraint)	Xu et al. (2017) [1]: Linear cost on p_j^{dis}	Xu + Collath et al. (2023) [1]: Adds SOS2 on $e_{soc}(t)$
Computational Cost	Trivial	Light-MILP / LP-Friendly	Heavy-MILP (Adds T sets of SOS2 variables) [1]
Key Trade-off	Pro: Simple. Con: Sub-optimal profit.	Pro: Captures 80% of degradation with 20% of computation. Con: Ignores calendar aging.	Pro: Highest physical accuracy. Con: Computationally intractable.
Strategic Verdict	Obsolete	Recommended Competition Model	Academically Correct, but Infeasible

2 Stage I: The Phase I Baseline – A Rigid Physical Constraint

The Phase I model established a strong foundation: a deterministic MILP capable of co-optimizing BESS bids across three distinct markets: the Day-Ahead (DA) Energy market, the FCR Capacity market, and the aFRR Capacity market [1, 1].

The sole representation of battery degradation within this model was a simple, rigid physical constraint: the "Daily Cycle Limit" [1, 1]. This constraint, referred to as Cst-5 in our internal project files, takes the form of a hard "wall":

$$\sum_{t \in \text{Day}} p_{dis}(t) \cdot \Delta t \leq N_{cycles} \cdot E_{nom} \quad (1)$$

Where N_{cycles} (e.g., 1.0, 1.5, or 2.0) is an arbitrary, pre-defined limit on daily throughput.[1]

2.1 Trade-off Analysis (Stage I)

- **Pro:** This constraint is computationally trivial. It is a simple linear sum that adds no complexity to the MILP solver.
- **Con:** This constraint is "economically blind." It is a blunt instrument that treats all energy throughput identically. It cannot differentiate between a high-profit cycle (e.g., an arbitrage spread of 200 EUR/MWh) and a low-profit, marginal cycle (e.g., a 20 EUR/MWh spread).

The optimizer, seeking to maximize revenue, will execute cycles in descending order of profitability until it hits this arbitrary "wall," at which point it is forced to stop all discharging operations. This inflexibility is the model's critical weakness. It inevitably leaves substantial, uncaptured profits on the table whenever the hard limit is reached before the market opportunities are exhausted.

This analysis reveals the primary logical and strategic imperative for the Phase II model. To achieve true economic optimality, this physical wall must be deconstructed. The concept of a "limit" must be transformed into a "price." The optimizer must be allowed to make an economic trade-off: to *choose* to "buy" an additional cycle, if and only if, the market revenue from that cycle exceeds the monetized cost of the battery lifetime consumed.

3 Stage II: Evolving to an Economic Framework (Model ii: Xu-Only)

Stage II represents the first and most significant logical leap: transforming the model from a physically-constrained profit-maximizer to a true bi-objective economic optimizer. This is achieved by redefining the objective function and introducing a flexible, cost-based model for the dominant degradation driver.

3.1 The New Objective: The Bi-Objective Weighted Sum

The entire optimization problem is reformulated as a bi-objective "Weighted Sum Method," as defined in our Phase II mathematical formulation.[1] The new objective is:

$$\max Z = \mathbb{P}^{\text{Markets}} - \alpha \cdot C^{\text{Degradation}} \quad (2)$$

Where:

- $\mathbb{P}^{\text{Markets}}$ represents the sum of all market revenues (from DA, FCR, aFRR Capacity, and the new aFRR Energy market).
- $C^{\text{Degradation}}$ is the total monetized cost of battery degradation.
- α is the "degradation price," a critical meta-parameter that serves as the "exchange rate" between 1 EUR of profit today and 1 EUR of battery lifetime cost.[1]

This formulation directly pits short-term revenue against long-term asset health, forcing the optimizer to find the optimal balance. The task of finding the "correct" α is handled by our Meta-Optimization framework (detailed in Section 5).

3.2 Modeling the Dominant Driver: Cyclic Aging (Xu et al., 2017)

For a BESS operating in high-frequency arbitrage and ancillary service markets, the dominant degradation driver is *cyclic aging*. This mechanism is highly non-linear: the damage (and thus cost) caused by a discharge is a convex function of its Depth-of-Cycle (DOC). A single deep discharge from 100% to 20% SOC is disproportionately more damaging than multiple, shallow 10% SOC cycles.[1]

To model this, we implement the "Xu-Only" model, derived from our Phase II formulation (Section 2.2.1) [1] and based on the methodology proposed by Xu et al. (2017).[1]

This method linearizes the non-linear cost by creating a set of J virtual energy segments within the battery, each with its own state variable $e_{\text{soc},j}(t)$.

1. **Segmented SOC Dynamics:** The battery's total SOC is the sum of these virtual segments, each with a defined capacity E_j^{seg} . [1]

$$e_{\text{soc}}(t) = \sum_{j \in J} e_{\text{soc},j}(t) \quad (\text{Eq. 8}) \quad (3)$$

$$0 \leq e_{\text{soc},j}(t) \leq E_j^{\text{seg}} \quad (\text{Eq. 14}) \quad (4)$$

2. **Segmented Power Flow:** The total discharge power is now the sum of power drawn from these segments.[1]

$$p_{\text{dis}}^{\text{total}}(t) = \sum_{j \in J} p_j^{\text{dis}}(t) \quad (\text{Eq. 10}) \quad (5)$$

3. **Linearized Cost Function:** Each segment j is assigned a *marginal cyclic degradation cost*, c_j^{cost} , which increases with the depth of the segment (i.e., $c_1 < c_2 < \dots < c_J$). The total cyclic degradation cost is the sum of power discharged from each segment, multiplied by its specific cost.[1]

$$C^{\text{cyc}} = \sum_{t \in T} \sum_{j \in J} \left(c_j^{\text{cost}} \cdot \frac{p_j^{\text{dis}}(t)}{\eta_{\text{dis}}} \cdot \Delta t \right) \quad (\text{Eq. 5}) \quad (6)$$

3.3 Stage II Trade-off Analysis

- **Pro:** This model is "economically smart." The optimizer is no longer blind to the cost of its actions. It can now dynamically compare the market revenue $\mathbb{P}^{\text{Markets}}$ against the degradation cost C^{cyc} . It will *only* execute a deep, high-cost discharge (using p_j^{dis} from an expensive, deep segment j) if the market price is high enough to justify the $\alpha \cdot C^{\text{cyc}}$ penalty.
- **Pro (The Key Computational Advantage):** The "Xu" methodology is computationally brilliant and is the core of our "Model (ii)" strategy.[1] It models a non-linear, convex cost function using *purely linear* variables and constraints. Because the objective is to maximize Z (and thus minimize C^{cyc}), and since $c_1 < c_2 < \dots < c_J$, the LP solver will *naturally* and *always* discharge from the cheapest available segment $j = 1$ before *ever* touching the more expensive segment $j = 2$. This inherent optimization behavior provides a perfect piecewise-linear approximation of the non-linear cost curve *without* requiring any binary variables or complex Special Ordered Sets (SOS2) constraints for *this specific mechanism*.[1] This keeps the problem "LP-friendly" and computationally fast.
- **Con:** This model is incomplete. It perfectly captures the cost of *using* the battery (cyclic aging, a "flow-dependent" cost) but entirely ignores the cost of *storing* energy (calendar aging, a "state-dependent" cost).

This analysis reveals a fundamental dichotomy in degradation modeling. The Xu method is elegant for *flow-dependent* costs (a cost on p_j^{dis}), where the optimizer can *choose* which flow (cheap p_1 or expensive p_2) to use. Calendar aging, however, is a non-linear function of the *state* variable $e_{\text{soc}}(t)$ [1, 1]. At any time t , the state has only *one* value; the optimizer has no "choice." Therefore, the simple LP trick cannot be used, and a new, more computationally complex technique is required to advance to Stage III.

4 Stage III: The Academically Complete Formulation (Model iii: Xu + Collath)

Stage III represents the full mathematical formulation as documented in our `p2_bi_model_ggdp.tex` file.[1] It integrates the "Xu-Only" model with the two remaining Phase II components: the aFRR Energy Market and the calendar aging cost model.

4.1 Integrating the Full Phase II Feature Set

- **Component A: aFRR Energy Market:** This is a straightforward, linear extension to our revenue streams.[1]

- **New Variables:** We introduce $p_{aFRR,E}^{\text{pos}}(t)$ (discharge) and $p_{aFRR,E}^{\text{neg}}(t)$ (charge).
- **New Profit Term:** A new profit term $\mathbb{P}^{aFRR_E}$ is added to the objective [1]:

$$\mathbb{P}^{aFRR_E} = \sum_{t \in T} \left(\frac{P_{aFRR,E}^{\text{pos}}(t)}{1000} p_{aFRR,E}^{\text{pos}}(t) - \frac{P_{aFRR,E}^{\text{neg}}(t)}{1000} p_{aFRR,E}^{\text{neg}}(t) \right) \Delta t \quad (\text{Eq. 4})$$

- **Logic:** This requires updating all physical constraints to use new "total" power variables (e.g., $p_{\text{ch}}^{\text{total}}(t) = p_{\text{ch}}(t) + p_{aFRR,E}^{\text{neg}}(t)$).[1] This extension adds linear variables and constraints, which has a negligible impact on computational complexity.
- **Component B: Calendar Aging (Collath et al., 2023):** This is the second, computationally significant, degradation component.[1]

- **Problem:** Calendar aging is the slow, time-based degradation of the battery, which is non-linearly accelerated by being held at a high State of Charge (SOC).[1]
- **New Cost Term:** We introduce a new cost term, C^{cal} , to the objective function, which is a function of the *state* $e_{\text{soc}}(t)$. [1]

$$C^{\text{cal}} = \sum_{t \in T} c_{\text{cost}}^{\text{cal}}(t) \cdot \Delta t \quad (\text{Eq. 6})$$

4.2 The Computational Cost of Precision: The SOS2 Implementation

As identified in Stage II, we cannot use the simple "Xu" trick for this *state-dependent* cost. The cost $c_{\text{cost}}^{\text{cal}}(t)$ is a non-linear function of the state $e_{\text{soc}}(t)$. To linearize this function, we must use a standard, but computationally heavy, MILP technique: **Special Ordered Sets of Type 2 (SOS2)**. [1] This approach is consistent with the literature, such as Collath et al.. [1]

The implementation, as defined in our `.tex` file (Section 3.3.9), is as follows [1]:

1. **Define Breakpoints:** We first pre-calculate I breakpoints ($SOC_i^{\text{point}}, Cost_i^{\text{point}}$) from the non-linear curve provided in the literature. [1]
2. **Introduce SOS2 Variables:** We introduce a new set of continuous variables, $\lambda_{t,i}$, for each time step t and each breakpoint i .
3. **Convex Combination:** We then express the state $e_{\text{soc}}(t)$ and its corresponding cost $c_{\text{cost}}^{\text{cal}}(t)$ as a convex combination of these breakpoints, using the $\lambda_{t,i}$ variables as weights:

$$e_{\text{soc}}(t) = \sum_{i \in I} \lambda_{t,i} \cdot SOC_i^{\text{point}} \quad \forall t \quad (\text{Eq. 29}) \quad (7)$$

$$c_{\text{cost}}^{\text{cal}}(t) = \sum_{i \in I} \lambda_{t,i} \cdot Cost_i^{\text{point}} \quad \forall t \quad (\text{Eq. 30}) \quad (8)$$

$$\sum_{i \in I} \lambda_{t,i} = 1 \quad \forall t \quad (\text{Eq. 31}) \quad (9)$$

4. **The SOS2 Constraint:** Finally, we apply the SOS2 constraint, which dictates that at most two *adjacent* $\lambda_{t,i}$ variables can be non-zero. [1]

$$\{\lambda_{t,i}\}_{i \in I} \text{ are SOS2 variables} \quad \forall t \quad (\text{Eq. 32}) \quad (10)$$

This set of constraints forces the solution $(e_{\text{soc}}(t), c_{\text{cost}}^{\text{cal}}(t))$ to lie on the line segments connecting the pre-calculated breakpoints, creating an accurate piecewise-linear approximation of the non-linear calendar aging cost.

4.3 Stage III Trade-off Analysis

- **Pro:** This "Model (iii) (Xu + Collath)" is the most physically accurate and "academically complete" formulation. It captures all four market revenue streams, the dominant *flow-dependent* (cyclic) degradation, and the secondary *state-dependent* (calendar) degradation.
- **Con:** The computational cost is severe. The "Xu" model added only linear variables. The "Collath" model, via its SOS2 implementation, adds a set of *integer-like* variables and constraints for every single time step in the optimization horizon. A 48-hour optimization (192 intervals) would require 192 *sets* of SOS2 variables. This fundamentally changes the problem, exploding the search space and dramatically increasing the MILP solve time.

5 Implementation Strategy: A Two-Stage Meta-Optimization

The mathematical formulation defines our "inner-loop" solver. However, the competition's objectives—maximizing a 10-Year ROI by balancing revenue (30%) and degradation (30%)—present a bi-objective problem that is computationally infeasible to solve naively.

5.1 The Computational Challenge: Infeasibility of Naive Meta-Optimization

The core task is to solve the bi-objective function [?]:

$$\max \mathbb{P} - \alpha \cdot C^{\text{Degradation}}$$

Finding the optimal trade-off parameter, α^* , requires a "meta-optimization" loop (i.e., an α -sweep) [?].

A naive implementation would run a full 365-day Rolling Horizon (MPC) simulation for *each* α value. As our analysis shows, this is computationally impossible:

- A 4-hour execution step ($T_{\text{exec}} = 4h$) requires $(365 \text{ days} \times 6 \text{ rolls/day}) = 2190$ MILP solves.
- Assuming a modest 30 seconds per solve, the total time for *one* α value is $2190 \times 30s \approx 1095$ minutes, or **over 18 hours**.
- This makes an α -sweep (e.g., 20+ values) across 5 countries intractable.

5.2 The Two-Stage Solution: Sampled α -Sweep and Full-Horizon Run

To solve this, we decouple the "search" for α^* from the "final execution" of the model.

5.2.1 Stage 1: Sample-Based Meta-Optimization (Finding α^*)

Instead of running the full 365-day simulation for every α , we find α^* using a computationally "cheap" proxy.

1. **Select Sample Data:** We select a small number of "representative" periods (e.g., 3-4 individual weeks) from the 2024 dataset that capture high volatility (winter), low volatility (summer), and average conditions.
2. **Run Fast α -Sweep:** We execute our MPC meta-optimization loop (sweeping 20-30 α values) *only* on these sample weeks.
3. **Extrapolate & Select:** We extrapolate the (Profit vs. Degradation) results from these samples to a full year, calculate the 10-Year ROI for each α , and select the α^* that yields the highest *extrapolated* ROI.

This "sample-based" search can be completed within our 2-hour computational budget.

5.2.2 Stage 2: Full-Year Rolling Horizon (Final Run)

Once the *single* optimal α^* is identified in Stage 1, we perform *one* high-fidelity, full 365-day MPC simulation to generate our final submission files. For this single run, we optimize the MPC window parameters for accuracy and speed:

- **Execution Step (T_{exec}): 24 hours (1 roll per day).**
 - **Justification:** This is our primary time-saving measure. It reduces the total MILP solves from ~ 2190 (for a 4h-roll) to just **365 solves**. Given that the competition provides deterministic data (perfect foresight), the main benefit of a shorter rolling window—managing uncertainty—is not applicable.

- **Planning Horizon ($T_{horizon}$): 48 hours (2-day lookahead).**

- **Justification:** A 24-hour horizon is "myopic" and will make poor decisions at the end of the day (e.g., failing to prepare for the next morning's price changes). A 48-hour horizon ensures the model can see at least one full 24-hour cycle ahead, allowing it to optimize across daily market cycles effectively.

This two-stage strategy allows us to find the optimal profit-degradation trade-off (α^*) in a feasible timeframe (Stage 1) and then generate an accurate, high-quality result using that parameter (Stage 2).

5.3 Solving the Bi-Objective Problem: The α -Sweep

- Explain that the 30%/30% (Revenue/Degradation) weighting makes this a bi-objective problem [?].
- Our solution is the **Weighted Sum Method**: $\max \left(\text{Profit} - \alpha \cdot (\text{Degradation Cost}) \right)$.
- The "magic number" α is our tuning knob. We find the *best* α^* by running a **meta-optimization loop**: We run the *entire* 365-day MPC simulation for many different α values and pick the one that gives the **maximum 10-Year ROI**.

6 Strategic Justification: Why Model (ii) is the Superior Competition Choice

The previous sections have defined the *what* (the three model stages) and the *how* (the meta-algorithm). This final section provides the strategic *why*—a justification for our selection of Model (ii) (Xu-Only) as the optimal and only feasible choice for this competition.

6.1 The Compounding Computational Burden

The true computational challenge is not the time to solve *one* 48-hour MILP. The true challenge is the *total* time required to complete the *entire meta-optimization*. [1] The total computational time is:

$$T_{Total} = (N_{\alpha_values}) \times (N_{MPC_solves_per_year}) \times (T_{solve_per_MILP})$$

A quantitative analysis reveals the staggering scale of this challenge:

- N_{α_values} : A reasonably granular parameter sweep to find the optimal α requires at least **30 values**.
- $N_{MPC_solves_per_year}$: A 365-day simulation, using a 4-hour execution block (rolling 6 times per day), requires $(365 * 6) = \mathbf{2,190 \text{ individual MILP solves per year}}$.
- **Total MILP Solves**: $30 (\text{alphas}) \times 2,190 (\text{solves/alpha}) = 65,700 \text{ individual MILP solves}$.

To find the 10-Year ROI champion, our chosen model must be solved, back-to-back, over 65,000 times. This is the metric upon which our model choice *must* be based.

6.2 The "Open-Source Solver" Constraint

The Phase II documentation abstract explicitly notes a critical competition constraint: "responding to the practical constraints of the competition (deterministic data, **open-source solvers**)". [1]

This is not a trivial detail. As optimization researchers, it is known that open-source solvers (e.g., CBC, GLPK) are orders of magnitude slower than their commercial counterparts (e.g., Gurobi, CPLEX) when handling complex MILPs, especially those with a large number of integer or SOS2

constraints. The reference paper for calendar aging, Collath et al., itself used the commercial solver Gurobi.[1]

This constraint acts as a powerful filter, forcing a re-evaluation of the "Xu-Only" vs. "Xu + Collath" trade-off:

- **Model (ii) (Xu-Only):** The 48-hour (192-interval) MILP is computationally "light." The degradation model is linear.[1] The only integer/binary variables are those for market bidding (e.g., $y_{ch}(t)$). The $T_{solve_per_MILP}$ with an open-source solver is *very fast* (e.g., estimated < 10 seconds).
- **Model (iii) (Xu + Collath):** The 48-hour (192-interval) MILP is computationally "heavy." To *every solve*, we add **192 sets of SOS2 variables** [1]—one for each time step. This explodes the problem's combinatorial complexity. The $T_{solve_per_MILP}$ with an open-source solver becomes *very slow* (e.g., estimated 5-10 minutes, or potentially failing to converge at all).

6.3 The 80/20 Solution: Optimizing for Feasibility and Impact

This analysis leads to a stark, unavoidable conclusion:

- **Total Time (Model iii: Xu + Collath):**
65,700 solves $\times$ 5 min/solve = 328,500 minutes $\approx$ 228 days
This is computationally **INTRACTABLE** and unfeasible within the competition timeline.
- **Total Time (Model ii: Xu-Only):**
65,700 solves $\times$ 10 sec/solve = 657,000 seconds $\approx$ 7.6 days
This is computationally **FEASIBLE**.

The "academically complete" Model (iii) is a strategic trap. It pursues a secondary degradation effect (calendar aging) at the cost of an *exponential* increase in computational time, which is made fatal by the open-source solver constraint.[1]

Model (ii) (Xu-Only) is the superior strategic choice. It captures the *dominant* (80%) degradation driver (cyclic aging) [1] using a *computationally superior* (20%) method [1] that is perfectly suited for the 65,700-solve meta-optimization [1] we must perform. Model (ii) is not a compromise; it is the *correct* design for this specific engineering challenge, representing the optimal trade-off between physical realism and the practical, compounding computational constraints of the competition. It is the only model that allows us to *actually complete the meta-optimization* and find the true 10-Year ROI, which is the ultimate goal.